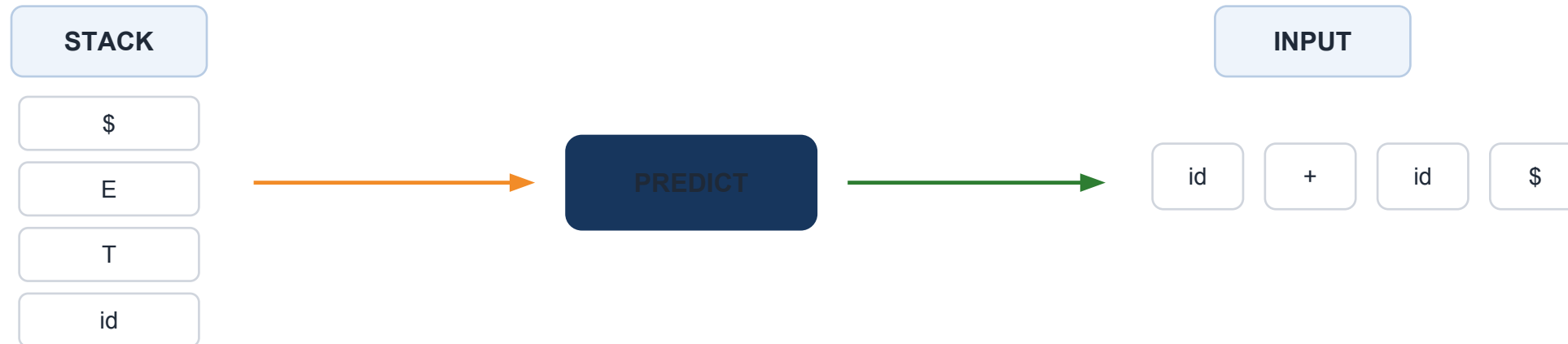


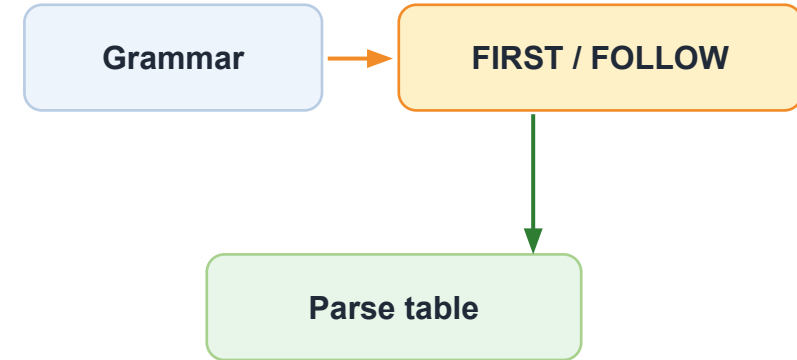
Top-Down Parsing

Aho–Ullman style lecture deck for UG CSE Compiler Design

50 editable slides: theory, algorithms, parse tables, examples, and exercises

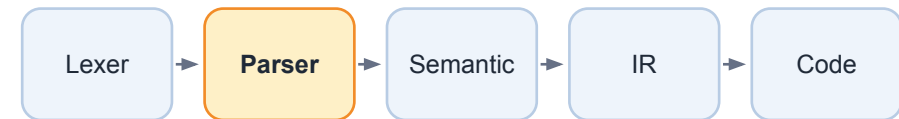


- Explain how top-down parsing constructs a leftmost derivation.
- Write recursive-descent procedures from grammar productions.
- Compute FIRST and FOLLOW sets for LL(1) parsing.
- Build and use a predictive parsing table.
- Transform grammars using left-recursion elimination and left factoring.



Outcome: students should be able to parse a small expression language by hand.

- Lexer groups characters into tokens.
- Parser checks token sequence against grammar.
- Parser output may be a parse tree, AST, or syntax-directed actions.
- Later phases perform semantic analysis and code generation.



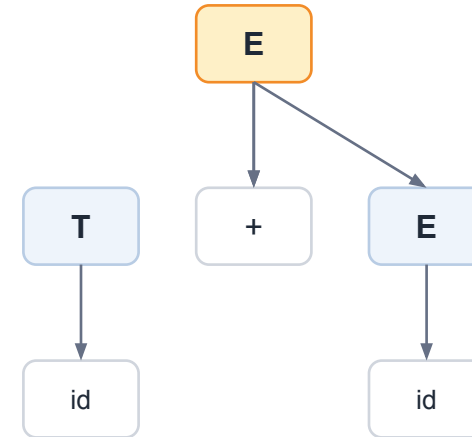
Parser checks whether token stream obeys grammar.

Parser is the syntax gatekeeper.

What “Top-Down” Means

04

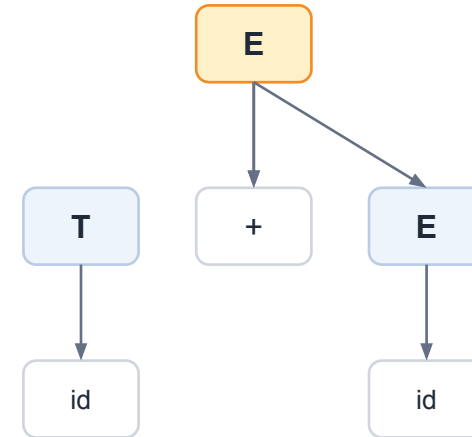
- Begin with the start symbol at the root.
- Expand nonterminals until the leaves match the input tokens.
- Usually corresponds to a leftmost derivation.
- The central action is prediction: which production should be used next?



Small grammar

```
E -> T E'  
E' -> + T E' | ε  
T -> id
```

- At every derivation step, expand the leftmost nonterminal.
- Top-down parsers naturally follow this discipline.
- The derivation sequence explains the shape of the parse tree.
- A successful derivation ends with the exact input string.



Example: id + id

```
E => T E'  
=> id E'  
=> id + T E'  
=> id + id
```

- Recursive descent uses procedures for nonterminals.
- General recursive descent may use backtracking.
- Predictive parsing avoids backtracking when grammar is LL(1).
- Table-driven LL(1) uses stack + input + parse table.



Procedure sketch

```
E() {  
    T();  
    Eprime();  
}  
match(id);
```

Predictive parsing is the practical focus for LL(1).

- Each nonterminal becomes a procedure.
- Terminals become `match(token)` calls.
- Alternatives become if/else or switch branches.
- The code often mirrors the grammar structure.



Procedure sketch

```
E() {  
    T();  
    Eprime();  
}  
match(id);
```

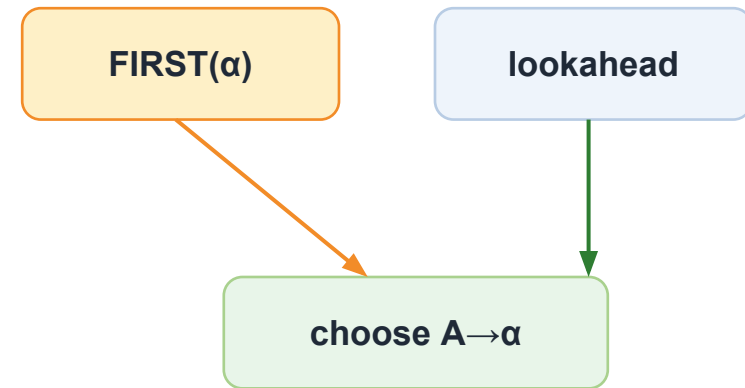
Procedure from $E \rightarrow T E'$

```
E() {  
    T();  
    Eprime();  
}
```

Example: Statement Grammar

08

- `stmt()` chooses by looking at the first token.
- Distinct starting tokens make prediction easy.
- `if`, `while`, `begin`, and `id` can select different productions.
- Problems appear when alternatives share prefixes.



Statement grammar fragment

```
stmt -> if expr then stmt else stmt
      | while expr do stmt
      | begin stmt_list end
      | id := expr
```

- For $A \rightarrow X_1 X_2 \dots X_n$, parse symbols from left to right.
- If X_i is a terminal, call `match( $X_i$ )`.
- If X_i is a nonterminal, call $X_i()$.
- Choosing the right alternative is the main challenge.



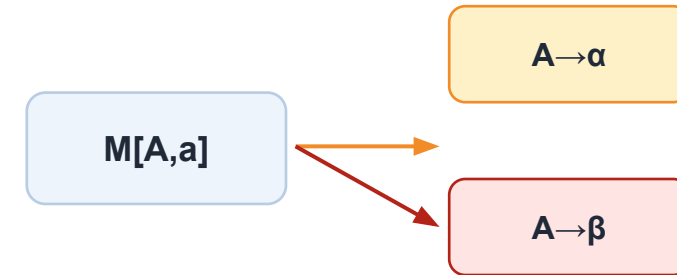
Procedure sketch

```
E() {  
    T();  
    Eprime();  
}  
match(id);
```

Recursive-descent template

```
procedure A():  
    choose production A -> X1...Xn  
    for Xi in RHS:  
        if terminal: match(Xi)  
        else: Xi()
```

- Backtracking tries one alternative and returns if it fails.
- It can parse more grammars than predictive parsing.
- However, it complicates implementation and may be inefficient.
- Compiler courses emphasize grammars that avoid backtracking.

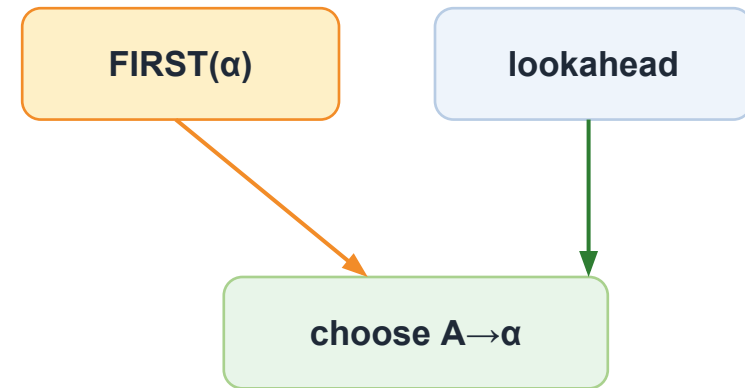


Both alternatives start with a

```
A -> a B | a C
B -> b
C -> c
```

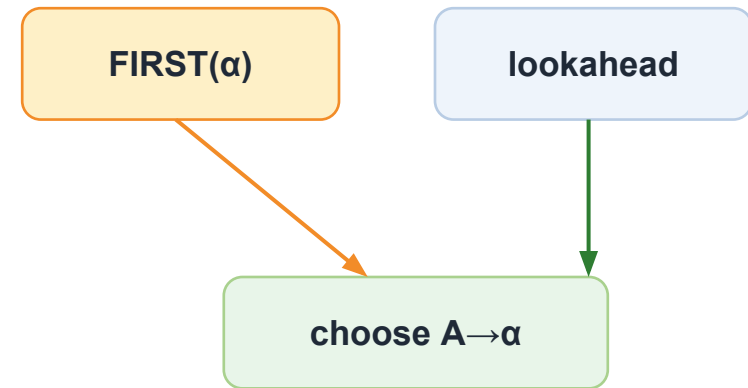
Two productions in one cell = not LL(1).

- A predictive parser never backtracks.
- Current nonterminal plus one lookahead token selects the production.
- LL(1) means Left-to-right input, Leftmost derivation, 1 lookahead.
- Success depends on grammar design.



LL(1) parsing is deterministic prediction.

- $\text{FIRST}(\alpha)$ is the set of terminals that can begin strings derived from α .
- If α can derive empty string, include ϵ .
- FIRST guides selection of ordinary productions.
- Use FIRST of the RHS, not only FIRST of the LHS nonterminal.



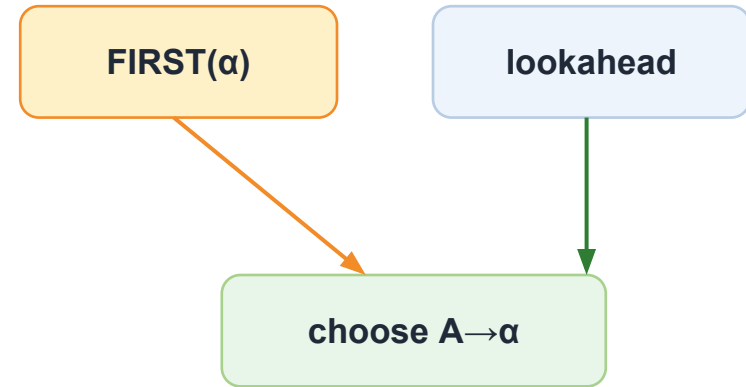
Example grammar

```
T  -> F T'  
T' -> * F T' |  $\epsilon$   
F  -> ( E ) | id
```

Computing FIRST: Rules

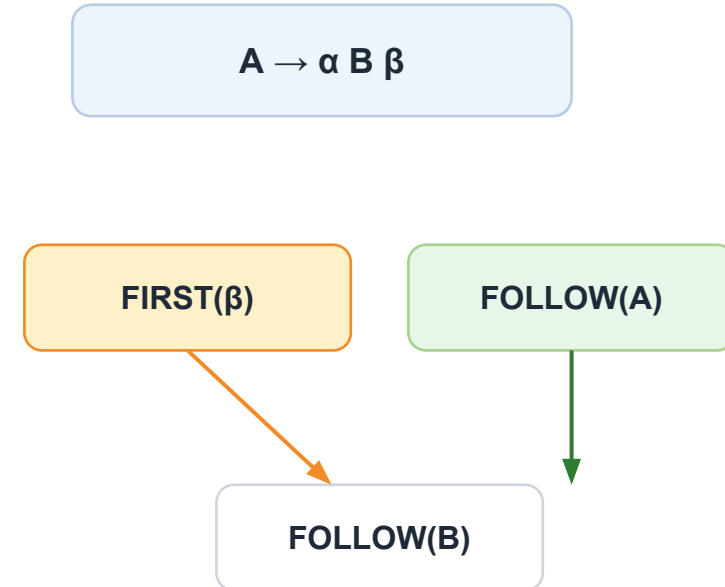
13

- If X is terminal, $\text{FIRST}(X) = \{X\}$.
- If $X \rightarrow \epsilon$, add ϵ to $\text{FIRST}(X)$.
- For $X \rightarrow Y_1Y_2\dots Y_k$, add $\text{FIRST}(Y_1)$ without ϵ .
- If Y_1 is nullable, inspect Y_2 , and continue.
- If all Y_i are nullable, add ϵ .



Stop when a symbol cannot derive ϵ .

- FOLLOW(A) contains terminals that may immediately follow A.
- Put \$ in FOLLOW(S) for the start symbol S.
- FOLLOW helps choose ϵ productions.
- FOLLOW sets are also useful for error recovery.



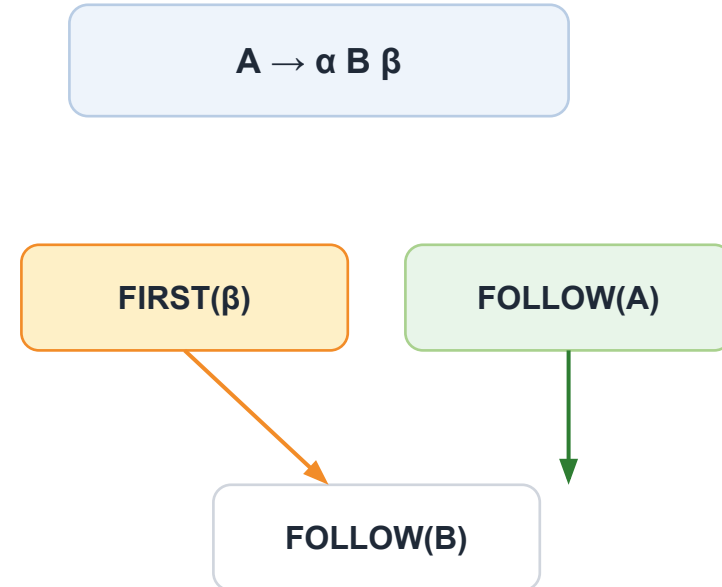
FOLLOW example context

```
E -> T E'  
E' -> + T E' |  $\epsilon$   
F -> ( E ) | id
```

Computing FOLLOW: Rules

15

- For $A \rightarrow \alpha B \beta$, add $\text{FIRST}(\beta)$ without ϵ to $\text{FOLLOW}(B)$.
- For $A \rightarrow \alpha B$ or nullable β , add $\text{FOLLOW}(A)$ to $\text{FOLLOW}(B)$.
- Repeat until no FOLLOW set changes.
- Always track the start symbol rule.



Question to ask: what can legally appear after B?

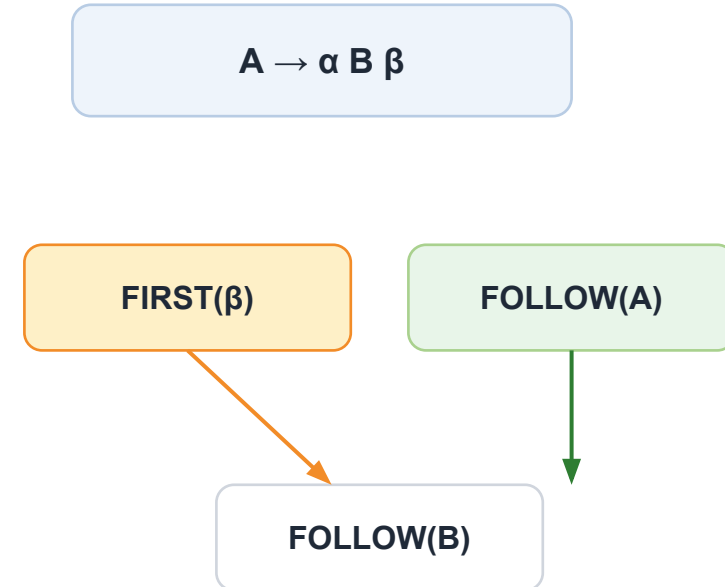
Nullable and Epsilon Productions

16

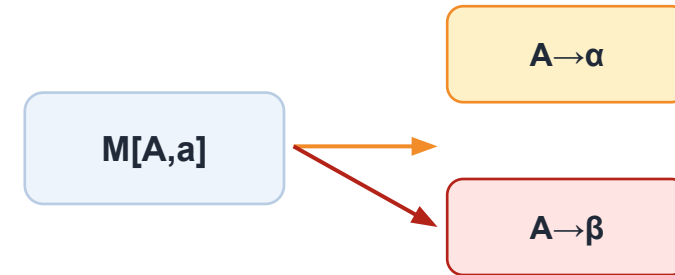
- A symbol is nullable if it can derive ϵ .
- Nullable symbols affect FIRST of longer strings.
- They also cause FOLLOW propagation.
- ϵ is not a token in the input stream.

Nullable example

```
A -> B C
B -> b |  $\epsilon$ 
C -> c |  $\epsilon$ 
FIRST(A) = {b, c,  $\epsilon$ }
```



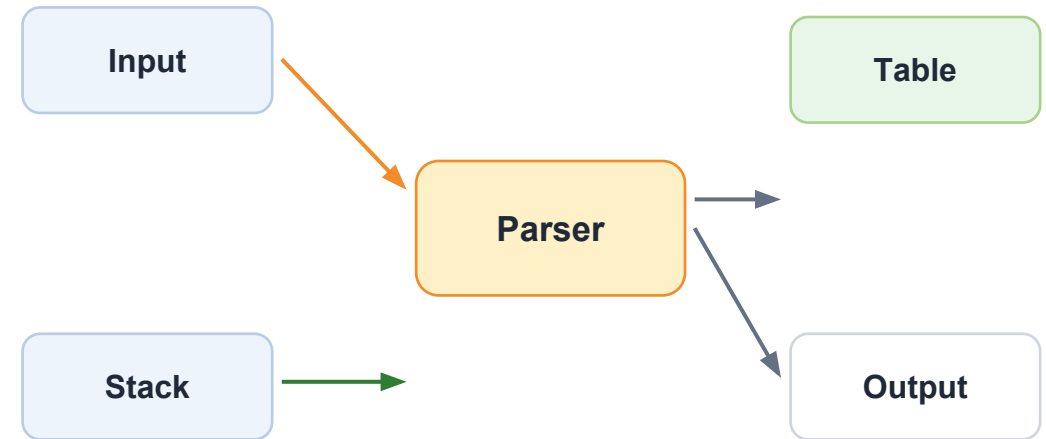
- For $A \rightarrow \alpha \mid \beta$, $\text{FIRST}(\alpha)$ and $\text{FIRST}(\beta)$ must be disjoint.
- If an alternative can derive ϵ , compare the other FIRST sets with $\text{FOLLOW}(A)$.
- Every table entry must contain at most one production.
- Conflicts mean the grammar is not LL(1).



Two productions in one cell = not LL(1).

LL(1) is a grammar property.

- Input buffer holds tokens ending with \$.
- Stack holds grammar symbols, with \$ at the bottom.
- Parsing table $M[A,a]$ maps nonterminal/lookahead to production.
- Output is the production sequence or an AST-building action.



Constructing the Parse Table

19

- For each production $A \rightarrow \alpha$, examine $\text{FIRST}(\alpha)$.
- For every terminal a in $\text{FIRST}(\alpha)$, put $A \rightarrow \alpha$ in $M[A, a]$.
- If ϵ is in $\text{FIRST}(\alpha)$, use every b in $\text{FOLLOW}(A)$.
- Blank cells are syntax-error entries.

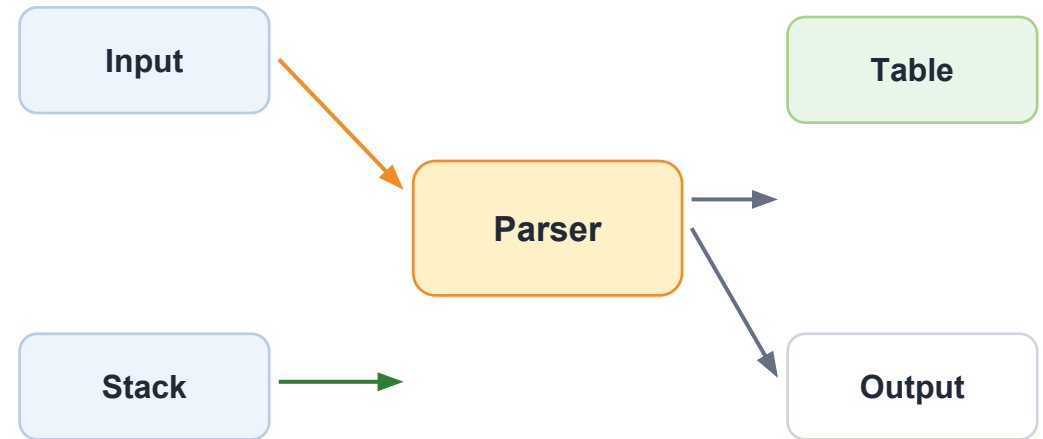
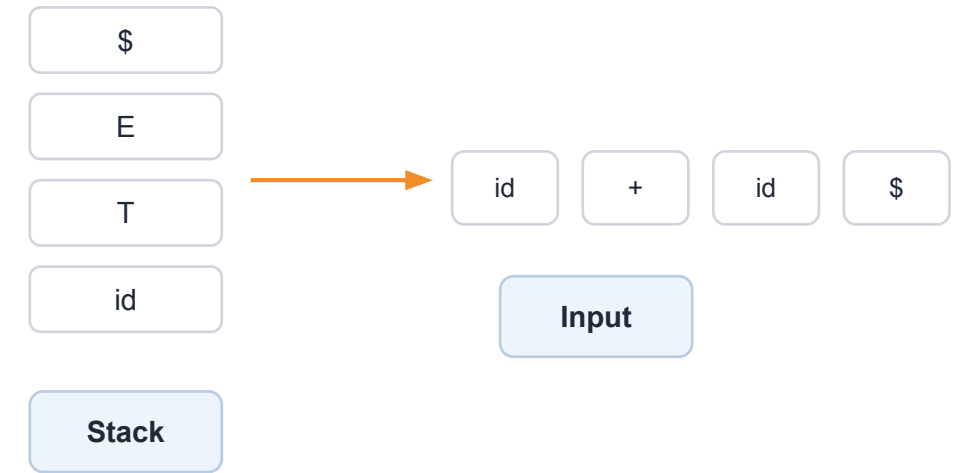


Table construction rule

```
if  $a \in \text{FIRST}(\alpha)$ :  $M[A, a] = A \rightarrow \alpha$   
if  $\epsilon \in \text{FIRST}(\alpha)$ :  
  for  $b \in \text{FOLLOW}(A)$ :  $M[A, b] = A \rightarrow \alpha$ 
```

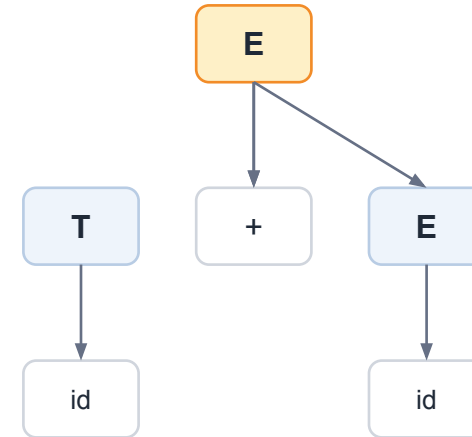
- Let X be top of stack and a be current input token.
- If X is a terminal and $X = a$, pop and advance.
- If X is a nonterminal, consult $M[X,a]$.
- Replace X by the RHS in reverse order.
- Accept when both stack and input have \$.



Core loop

```
while top != $:  
    X = stack.top(); a = lookahead  
    if terminal(X): match(X)  
    else: use M[X,a]
```

- This grammar removes left recursion from expression syntax.
- E handles addition-level expressions.
- T handles multiplication-level expressions.
- F handles identifiers and parenthesized expressions.

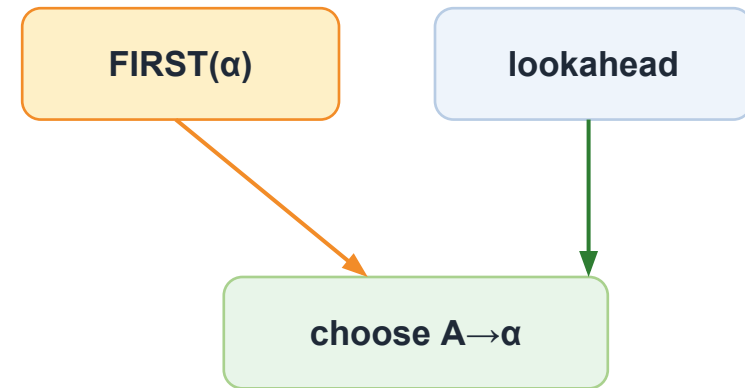


Expression grammar

```
E  -> T E'
E' -> + T E' | ε
T  -> F T'
T' -> * F T' | ε
F  -> ( E ) | id
```

- $\text{FIRST}(E)$ and $\text{FIRST}(T)$ are $\{ (, \text{id} \}$.
- $\text{FIRST}(E')$ is $\{ +, \epsilon \}$.
- $\text{FIRST}(T')$ is $\{ *, \epsilon \}$.
- FOLLOW sets decide when E' and T' should become ϵ .

Nonterminal	FIRST	FOLLOW
E	(, id	), \$
E'	+, ϵ	), \$
T	(, id	+,), \$
T'	*, ϵ	+,), \$
F	(, id	*, +,), \$



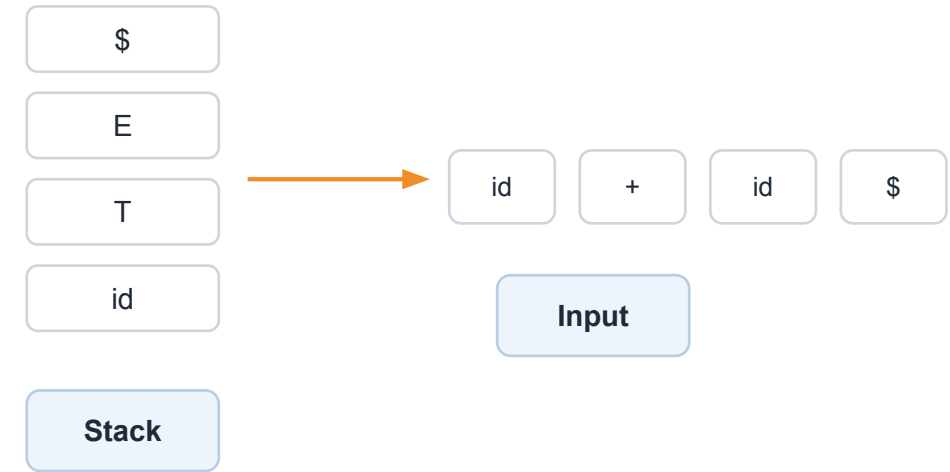
Predictive Table for Expression Grammar

23

	id	+	*	(	)	\$
E	$E \rightarrow T E'$			$E \rightarrow T E'$		
E'		$E' \rightarrow + T E'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow F T'$			$T \rightarrow F T'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow * F T'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

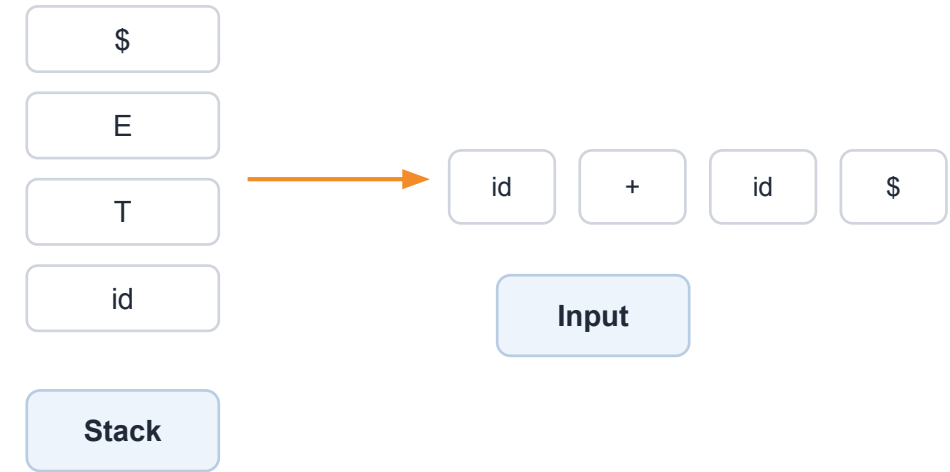
Filled cells are decisions; blank cells are errors.

- Trace begins with stack \$ E and input id + id * id \$.
- Use M[E,id] to expand $E \rightarrow T E'$.
- Use M[T,id] to expand $T \rightarrow F T'$.
- Use M[F,id] to match id.
- When lookahead is + and top is T', choose ϵ .



Step	Stack	Input	Action
1	\$ E	id + id * id \$	$E \rightarrow T E'$
2	\$ E' T	id + id * id \$	$T \rightarrow F T'$
3	\$ E' T' F	id + id * id \$	$F \rightarrow \text{id}$
4	\$ E' T' id	id + id * id \$	match id
5	\$ E' T'	+ id * id \$	$T' \rightarrow \epsilon$

- After + is matched, parse the second T.
- The lookahead * triggers $T' \rightarrow * F T'$.
- The final \$ triggers $T' \rightarrow \epsilon$ and $E' \rightarrow \epsilon$.
- Accept when the stack and input both reach \$.

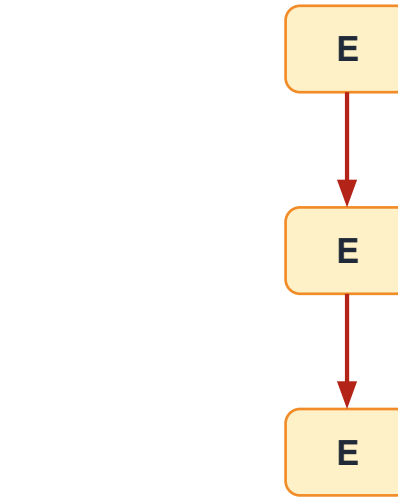


Step	Stack	Input	Action
6	\$ E'	+ id * id \$	$E' \rightarrow + T E'$
7	\$ E' T +	+ id * id \$	match +
8	\$ E' T	id * id \$	$T \rightarrow F T'$
11	\$ E' T'	* id \$	$T' \rightarrow * F T'$
15	\$	\$	accept

Why Left Recursion Breaks Top-Down Parsing

26

- Immediate left recursion has form $A \rightarrow A \alpha \mid \beta$.
- A recursive-descent procedure calls itself before consuming input.
- This causes infinite recursion.
- Left recursion must be removed for LL-style top-down parsing.



Infinite recursion before consuming input.

Problem

```
E -> E + T | T
```

`E()` calls `E()` before matching a token.

Eliminating Immediate Left Recursion

27

- Original form: $A \rightarrow A \alpha_1 \mid A \alpha_2 \mid \beta_1 \mid \beta_2$.
- Transformed form: $A \rightarrow \beta_1 A' \mid \beta_2 A'$.
- New continuation: $A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \epsilon$.
- The new grammar consumes β before repetition.

Standard transformation

```
A  -> β A'
A' -> α A' | ε
```



Infinite recursion before consuming input.

Example: $E \rightarrow E + T \mid T$

28

- Here $\alpha = + T$ and $\beta = T$.
- After transformation, E first parses T .
- E' parses zero or more $+T$ suffixes.
- This supports top-down parsing.

Transformation

```
Before:  $E \rightarrow E + T \mid T$   
After:   $E \rightarrow T E'$   
         $E' \rightarrow + T E' \mid \epsilon$ 
```



Infinite recursion before consuming input.

- Indirect left recursion occurs through a chain of nonterminals.
- Example: $A \rightarrow B a$ and $B \rightarrow A b$.
- General removal substitutes earlier nonterminals, then removes immediate left recursion.
- This is more mechanical than conceptual.

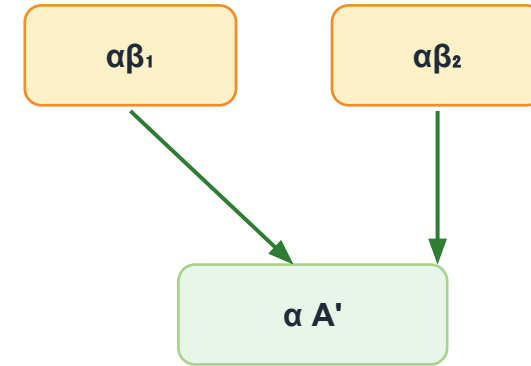
Indirect example

```
A -> B a | c  
B -> A b | d
```



Infinite recursion before consuming input.

- Left factoring removes common prefixes.
- It delays the decision until enough input has been seen.
- Original: $A \rightarrow \alpha \beta_1 \mid \alpha \beta_2$.
- Factored: $A \rightarrow \alpha A'$, $A' \rightarrow \beta_1 \mid \beta_2$.

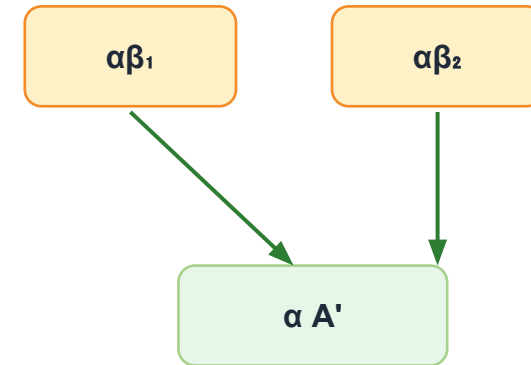


Factoring pattern

```
Before: A -> αβ1 | αβ2
After:  A  -> α A'
       A' -> β1 | β2
```

Read common prefix once, decide later.

- Two if alternatives share the prefix if E then stmt.
- Left factoring reads the common prefix once.
- The parser then chooses between else stmt and ϵ .
- Dangling-else ambiguity still requires a convention.

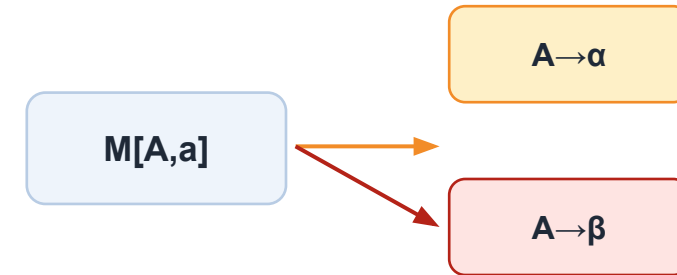


Factored form

```
stmt  -> if E then stmt stmt' | other
stmt' -> else stmt |  $\epsilon$ 
```

Read common prefix once, decide later.

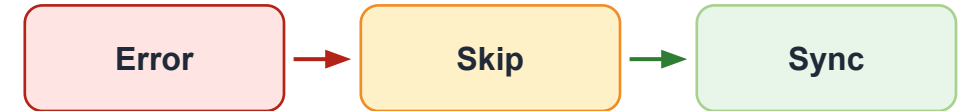
- FIRST/FIRST conflict: two alternatives begin with same lookahead.
- FIRST/FOLLOW conflict: ϵ alternative overlaps with a follower.
- Ambiguity can produce unavoidable conflicts.
- Conflicts require grammar redesign or a stronger parsing method.



Two productions in one cell = not LL(1).

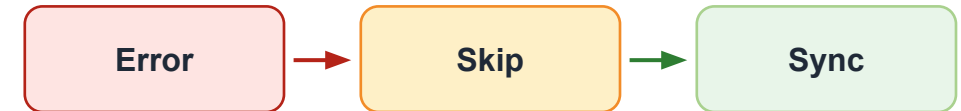
One table cell must not hold two productions.

- On error, discard input until a synchronizing token appears.
- FOLLOW(A) is often used as sync set for nonterminal A.
- The parser can then resume and report more errors.
- The goal is useful diagnostics, not perfect repair.



FOLLOW sets often provide synchronizing tokens.

- A blank table entry may call a recovery routine.
- The parser might insert a missing token or delete an extra token.
- Phrase-level recovery can produce clearer messages.
- But too much repair may hide the real error.



FOLLOW sets often provide synchronizing tokens.

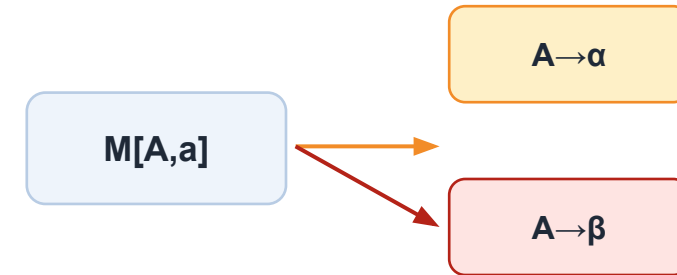
Recovery sketch

```
if M[A,a] is error:  
  report expected FIRST(A) or FOLLOW(A)  
  recover by skip / insert / sync
```

Grammars Not Suitable for LL(1)

35

- Left-recursive grammars are unsuitable.
- Common-prefix alternatives need factoring.
- Ambiguous grammars need disambiguation or redesign.
- Some languages need more than one lookahead token.

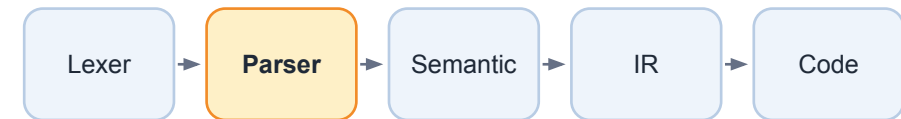


Two productions in one cell = not LL(1).

Changing grammar shape may affect semantic actions.

LL(1) vs Bottom-Up Parsing

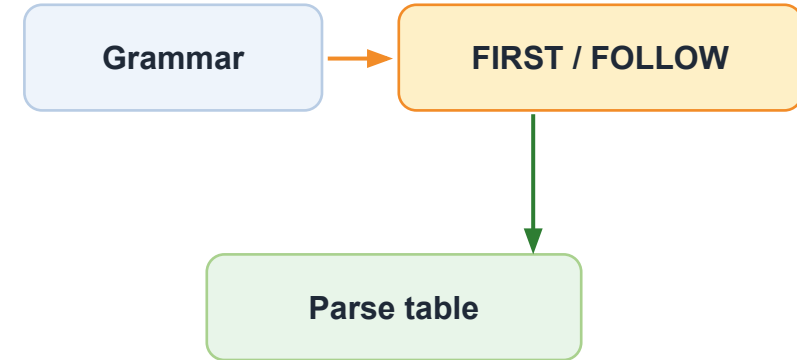
- LL(1) builds from root to leaves.
- Bottom-up parsing reduces leaves toward the root.
- LL(1) is easier to hand-code and teach.
- LR-style methods accept a larger grammar class.
- Both are important in compiler design.



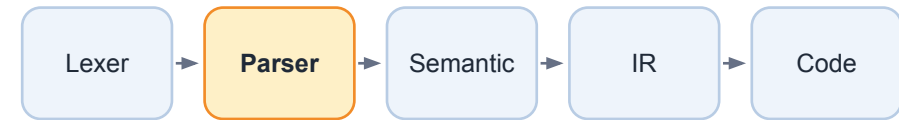
Aspect	LL(1)	LR-style
Direction	top-down	bottom-up
Implementation	recursive/table	shift-reduce
Coverage	smaller	larger

Parser checks whether token stream obeys grammar.

- Remove immediate and indirect left recursion.
- Left-factor common prefixes.
- Use separate nonterminals for precedence levels.
- Keep ϵ alternatives controlled by FOLLOW sets.
- Recheck FIRST/FOLLOW after every change.



- The parser reads tokens, not raw characters.
- The lexer hides whitespace and comments when appropriate.
- Lookahead is usually a token type.
- Token attributes carry lexemes, values, and locations.



Lexer to parser

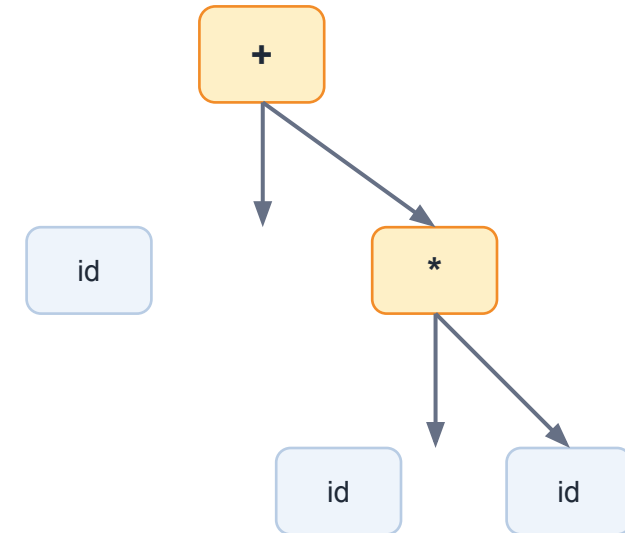
```
input chars → tokens → parser  
identifier("count") becomes token id with attribute count
```

Parser checks whether token stream obeys grammar.

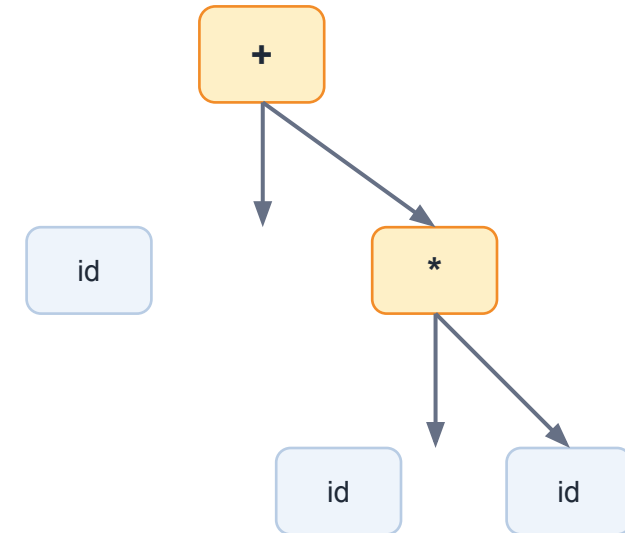
- Recursive descent can execute actions during parsing.
- Actions may build AST nodes.
- Actions may insert declarations into a symbol table.
- After grammar transformations, action placement may change.

Semantic note

`T()` returns `F` followed by `T'` continuation.
Inherited attributes help build left-associative ASTs.



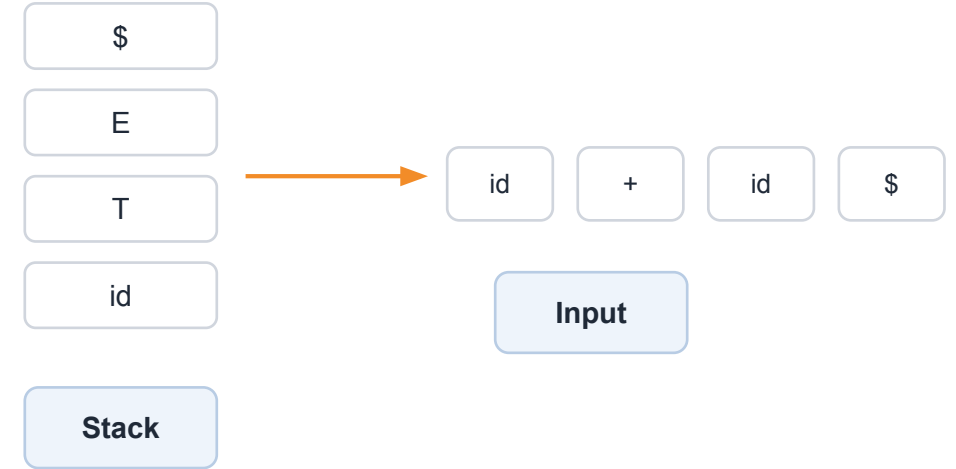
- Parse tree contains grammar details.
- AST keeps semantic structure only.
- Helper nonterminals like E' and T' may disappear in the AST.
- AST is the useful form for semantic analysis and code generation.



Implementing match()

41

- match() is the only routine that consumes input.
- It compares expected token with lookahead token.
- On success, it advances the lexer.
- On failure, it reports a clear syntax error.



Token match routine

```
match(expected):  
    if lookahead.type == expected:  
        lookahead = nextToken()  
    else:  
        error(expected, lookahead)
```

- E calls T and then E'.
- T calls F and then T'.
- F matches id or a parenthesized expression.
- The procedure structure mirrors the transformed grammar.



Procedure sketch

```
E() {  
    T();  
    Eprime();  
}  
match(id);
```

Expression parser sketch

```
E()      = T(); Eprime()  
Eprime() = if + then + T Eprime else ε  
T()      = F(); Tprime()  
Tprime() = if * then * F Tprime else ε  
F()      = id | ( E )
```

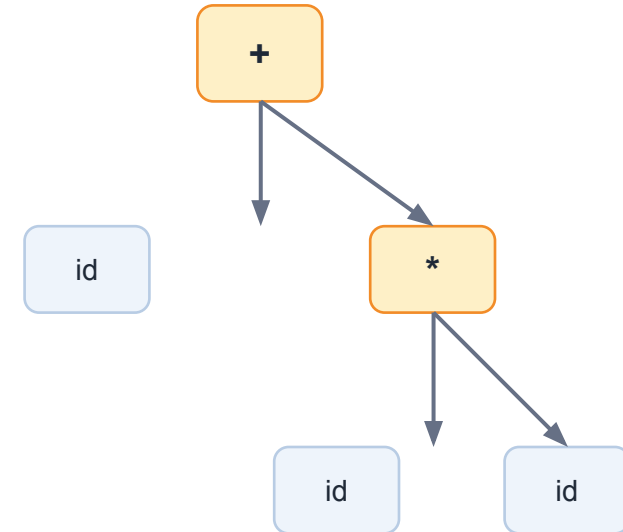
Handling Operator Precedence

43

- Place lower-precedence operators higher in the grammar.
- Place higher-precedence operators deeper in the grammar.
- Parentheses recursively call the expression nonterminal.
- This grammar structure avoids ambiguity in expressions.

Precedence by grammar levels

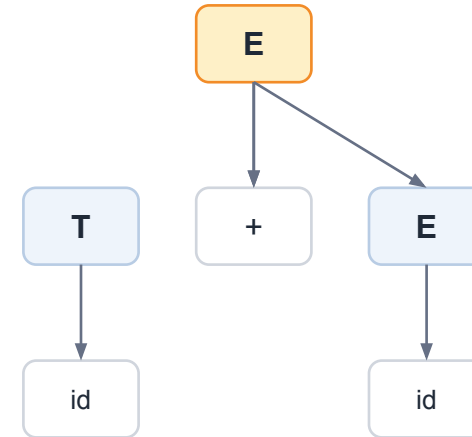
```
E -> T E'    handles +  
T -> F T'    handles *  
F -> (E) | id handles primaries
```



Case Study: Boolean Expressions

44

- not has highest precedence.
- and binds tighter than or.
- Each precedence level gets its own nonterminal.
- This pattern mirrors arithmetic expression parsing.



Boolean grammar

```
B -> C B'
B' -> or C B' | ε
C -> D C'
C' -> and D C' | ε
D -> not D | ( B ) | id
```

Manual Table Construction Exercise

45

- Compute FIRST sets for the grammar.
- Compute FOLLOW sets.
- Fill the LL(1) table.
- Identify whether any conflicts occur.
- Trace one valid string and one invalid string.

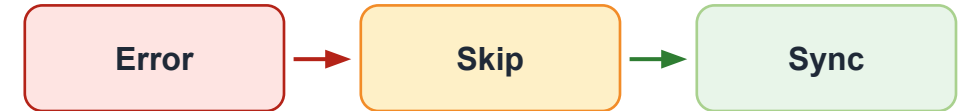
	a	b	c	\$
S				
A				
B				

Exercise grammar

```
S -> a A | b B
A -> c | ε
B -> d
```

Fill the blank LL(1) table.

- Putting ϵ in the input stream.
- Forgetting \$ in FOLLOW(start symbol).
- Using FIRST(A) instead of FIRST(α) for production $A \rightarrow \alpha$.
- Pushing the RHS in the wrong order.
- Ignoring action placement after grammar transformation.

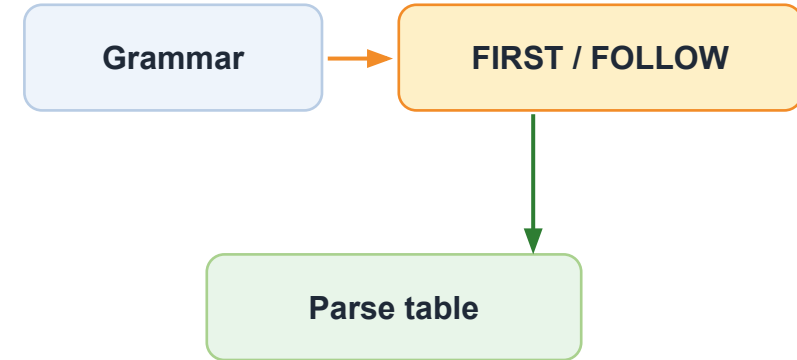


FOLLOW sets often provide synchronizing tokens.

Classroom Activity

47

- Group 1 transforms the grammar.
- Group 2 computes FIRST/FOLLOW sets.
- Group 3 builds the parsing table.
- Group 4 performs parsing traces.
- Group 5 designs recovery messages for error cells.



This makes parsing concrete and collaborative.

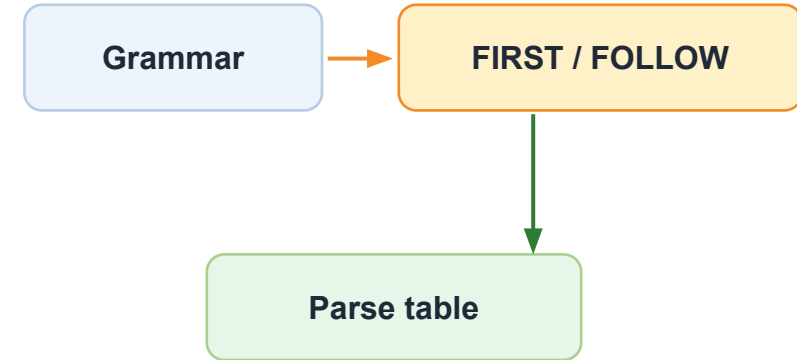
- What does LL(1) stand for?
- Why does left recursion cause infinite recursion in recursive descent?
- When do we use FOLLOW(A) while filling M[A,a]?
- What does a blank parsing-table entry mean?
- Why do we push the RHS in reverse order?

	a	b	c	\$
S				
A				
B				

Fill the blank LL(1) table.

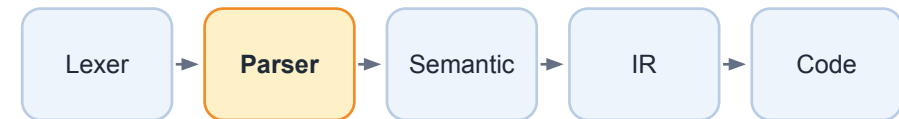
Use this as a 5-minute exit quiz.

- Top-down parsing builds from root to leaves.
- Recursive descent maps grammar to procedures.
- Predictive parsing uses one lookahead token and a parse table.
- FIRST guides normal choices; FOLLOW guides ϵ choices.
- Grammar transformations are essential for LL(1).



A good LL(1) parser is simple, deterministic, and easy to debug.

- Aho, Sethi and Ullman: Compilers — Principles, Techniques, and Tools.
- Aho and Ullman: Principles of Compiler Design.
- Practice with small grammars until FIRST/FOLLOW becomes routine.
- Next topic: bottom-up parsing and shift-reduce methods.



Parser checks whether token stream obeys grammar.

Thank you